

Université Paris Cité

UFR de Mathématiques et Informatique
Licence 2 Informatique — Projet L2G1

Conception générale :

Outil de traçabilité automatisé

Référent / Maîtrise d'ouvrage	M. Jeremy Meynadier
Équipe de développement	Emilie Huang Lamia Arrahmane Amel Kheloul Yseult Hellé Riglet
Référence document	D7 — Version A01
Date	Mars 2026

Table des matieres :

1. Préambule
2. Rappel des objectifs du projet
3. Architecture générale :
 - 3.1. Vue d'ensemble
 - 3.2. Choix techniques
4. Diagrammes :
 - 4.1. Diagramme d'utilisation
 - 4.2. Diagramme d'activité
5. Description des modules :
 - 5.1. Module 1 : Import & Parsing :
 - 5.1.1. Rôle
 - 5.1.2. Fonctionnement
 - 5.1.3. Gestion des erreurs
 - 5.2. Module 2 : Moteur de traçabilité :
 - 5.2.1. Rôle
 - 5.2.2. Fonctionnement
 - 5.2.3. Calcul du taux de couverture
 - 5.3. Module 3 : Interface graphique :
 - 5.3.1. Rôle
 - 5.3.2. Fonctionnalités
6. Interfaces en modules
7. Modèle de données :
 - 7.1. Structure Exigence
 - 7.2. Enumeration StatutTracabilite
 - 7.3. Structure LienBrise
 - 7.4. Structure RapportTracabilite
8. Prise en compte des contraintes non fonctionnelles :
 - 9.1. Performance
 - 9.2. Portabilité
 - 9.3. Confidentialité
 - 9.4. Maintenabilité
9. Glossaire
10. Références
11. Index
12. Suivi des modifications

1. Préambule :

Ce document est la Conception Générale du projet L2G1, développé dans le cadre de la Licence 2 d'Informatique et Applications à l'Université Paris Cité.

L'objectif de ce document est de définir l'architecture globale de l'outil de traçabilité. Il présente comment le système est organisé en modules, comment ces derniers communiquent entre eux, quelles données ils échangent, et quels sont les choix techniques utilisés pour répondre aux exigences du client.

Ce document s'adresse à l'équipe de développement, au référent pédagogique M. Jeremy Meynadier, et à tout lecteur souhaitant comprendre la structure générale du projet avant d'en consulter les détails techniques.

2. Rappel des objectifs :

Le projet L2G1 a pour but de développer un outil permettant de vérifier automatiquement la traçabilité des exigences dans un projet informatique.

Un projet est décrit à travers trois types de documents : les spécifications client (SSS), les spécifications projet (SRS) et le document de conception (SDD). L'outil doit s'assurer que chaque exigence du client est bien prise en compte à chaque niveau.

De façon plus précise, l'outil doit :

13. Importer des fichiers d'exigences aux formats .docx, .csv, .xls, .xlsx et .doc.
 - Détecter automatiquement les identifiants d'exigences dans ces fichiers grâce à des expressions régulières.
 - Établir les liens de traçabilité entre les niveaux SSS → SRS → SDD, et vérifier leur cohérence.
14. Identifier les exigences non couvertes (aucun lien), orphelines (SRS/SDD sans référence au niveau supérieur) et les liens brisés (référence vers un identifiant inexistant).
15. Calculer le taux de couverture global, les taux intermédiaires SSS→SRS et SRS→SDD, ainsi que le taux de traçabilité complète SSS→SRS→SDD.
 - Afficher les résultats dans une interface graphique interactive avec code couleur.

L'outil est développé exclusivement en C++ et fonctionne intégralement en local, sans connexion réseau. Il doit être compatible avec les systèmes Windows, Linux et macOS. Toutes les bibliothèques utilisées sont open source.

3. Architecture générale :

3.1. Vue d'ensemble

L'outil est structuré en trois modules indépendants qui communiquent de façon séquentielle. Chaque module a une responsabilité définie et ne connaît pas les détails internes des autres.

Cette séparation est un principe fondamental de la conception modulaire : elle facilite les tests, la maintenance et la répartition du travail dans l'équipe.

Le tableau suivant résume le rôle, les entrées et les sorties de chacun des trois modules :

Module	Rôle principal	Entrée	Sortie
Module 1 – Import & Parsing	Lire et analyser les fichiers sources, extraire les exigences et leurs métadonnées	Fichiers .docx / .csv / .xls / .xlsx / .doc	Vecteurs ExigenceSSS, ExigenceSRS, ExigenceSDD via ConvertisseurExigences
Module 2 – Moteur de traçabilité	Analyser les liens SSS→SRS→SDD, détecter anomalies, calculer les taux	Vecteurs d'objets typés produits par M1	RapportTracabilite (taux, liens brisés, orphelines, taux complet)
Module 3 – Interface graphique (Qt)	Afficher les résultats, permettre le filtrage interactif et l'export	RapportTracabilite produit par M2	Affichage : matrices, graphe, exports CSV / HTML / PDF

Tableau 1 – Vue d'ensemble des trois modules et de leurs échanges de données.

Ce tableau montre que les données circulent toujours dans le même sens : de M1 vers M2 sous forme d'une liste d'objets Exigence, puis de M2 vers M3 sous forme d'un objet rapport de traçabilité. M3 peut également interroger M2 lorsque l'utilisateur applique des filtres (interface I3, voir section 6).

3.2. Choix techniques

Le tableau ci-dessous présente les technologies utilisées pour chaque composant de l'outil. Ces choix respectent les contraintes du cahier des charges : utilisation exclusive du C++, bibliothèques open source, compatibilité multiplateforme et fonctionnement 100 % local.

Composant	Choix	Justification
Langage principal	C++	Imposé par le cahier des charges (EX-02).
Lecture .docx	libzip + tinyxml2	Un .docx est une archive ZIP. libzip le décompresse, tinyxml2 parse word/document.xml pour extraire le texte des balises <w:t>.
Lecture .xlsx	libzip + XML	Un .xlsx est aussi une archive ZIP. Les données sont dans xl/worksheets/, les chaînes dans xl/sharedStrings.xml.
Conversion .doc / .xls	Appel LibreOffice	Les formats binaires .doc et .xls sont convertis en .docx/.xlsx via LibreOffice avant extraction.
Lecture .csv	STL + parsing manuel	Les CSV sont des fichiers texte. La STL C++ suffit. Le délimiteur est auto-déecté.
Interface graphique	Qt	Capacités graphiques, support threads (QTimer), richesse des widgets (QTableWidget, QProgressBar, QPrinter...).
Expressions régulières	std::regex (C++11)	Intégré nativement, aucune dépendance externe.
Gestion des threads	std::thread + std::atomic<bool>	L'analyse tourne dans un thread séparé (ThreadManager). État communiqué via atomic<bool> sans mutex.
Gestion de versions	SVN	Outil requis par l'université.

Tableau 2 – Choix technologiques et justifications.

4. Diagrammes :

4.1. Diagramme d'utilisation

Les diagrammes de cas d'utilisation ci-dessous modélisent les interactions entre l'utilisateur et les fonctionnalités de l'outil.

Ils sont présentés séparément, un par module, afin de faciliter la lecture et d'identifier clairement les responsabilités de chaque composant. Ils ne montrent pas comment les choses se passent en interne, mais ce que l'utilisateur peut faire avec le système à chaque niveau.

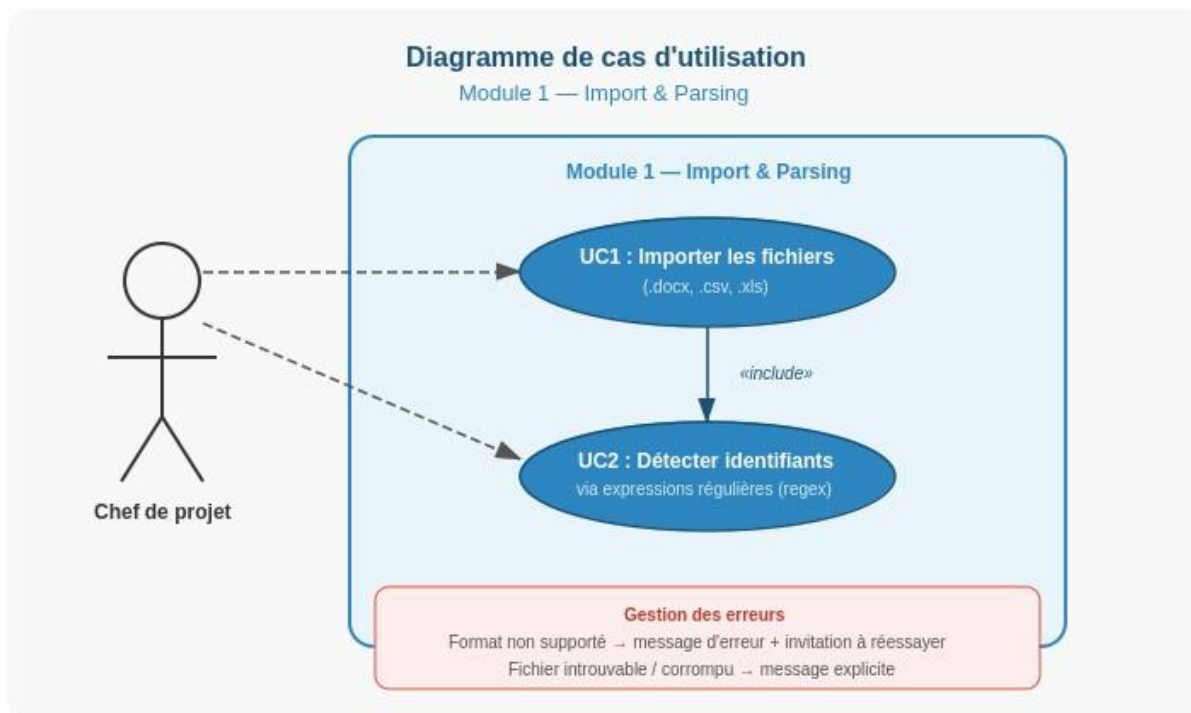


Figure 1 – Diagramme de cas d'utilisation - Module 1

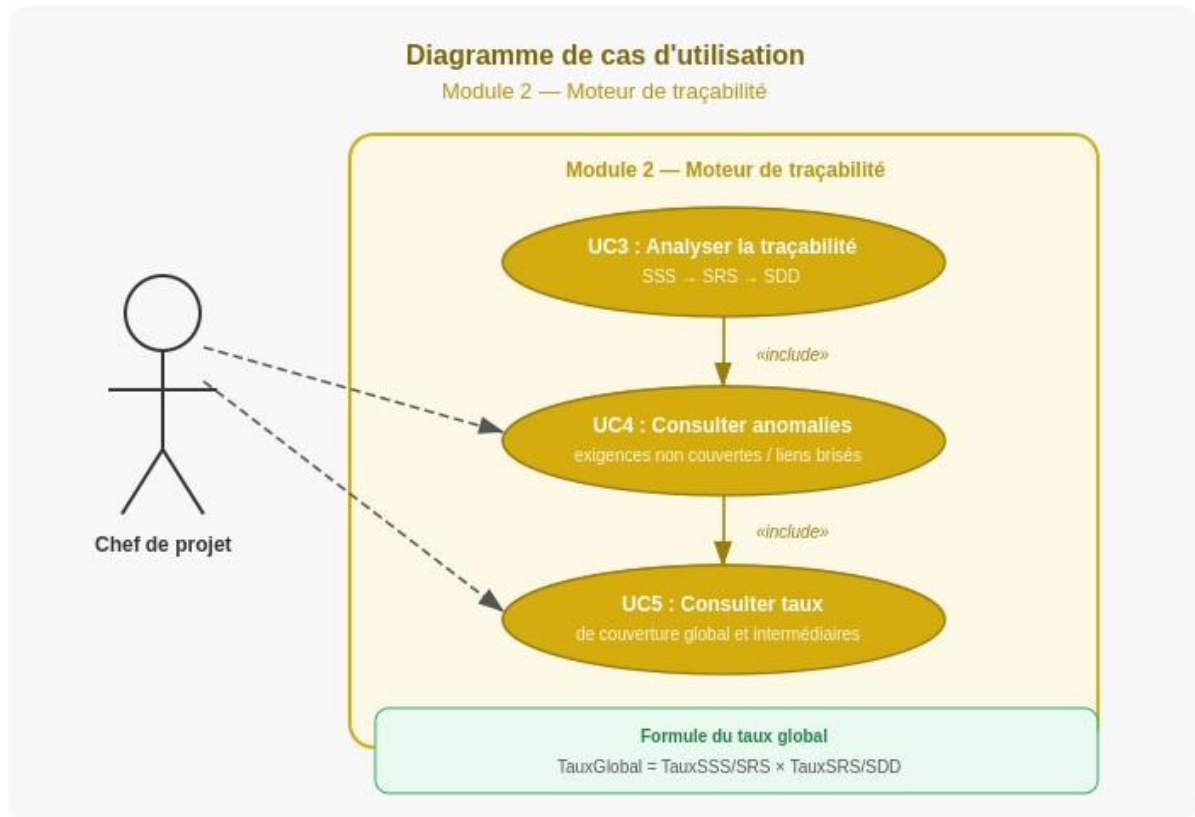


Figure 2 – Diagramme de cas d'utilisation - Module 2

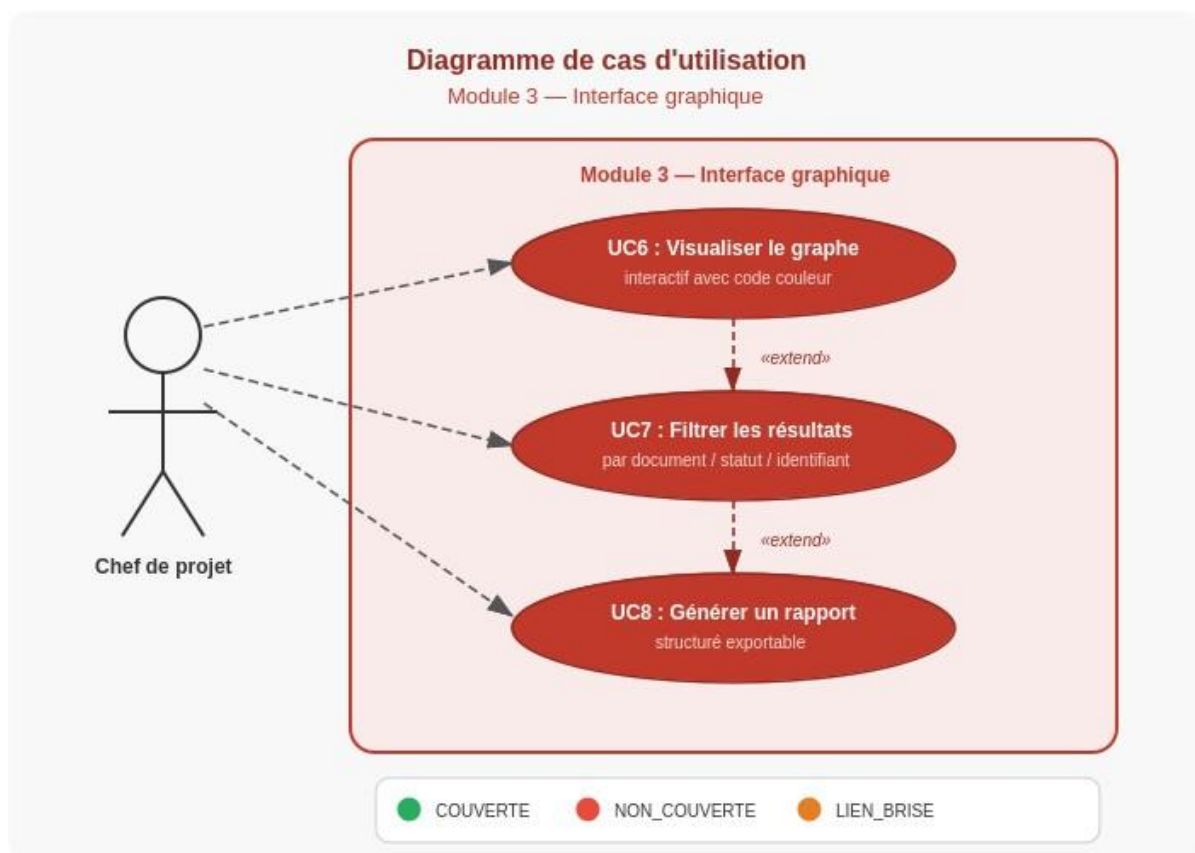


Figure 3 – Diagramme de cas d'utilisation - Module 3

Chaque diagramme fait apparaître un seul acteur : l'utilisateur.

C'est lui qui lance toutes les interactions avec l'outil. Les cas d'utilisation sont les suivants:

UC	Intitulé	Description	Module
UC 1	Importer les fichiers	L'utilisateur sélectionne les fichiers SSS, SRS et SDD (.docx, .csv, .xls, .xlsx, .doc). Plusieurs fichiers par niveau.	M1
UC 2	Détecter les identifiants	L'outil applique le pattern regex (configurable par fichier) pour extraire les identifiants et leurs métadonnées.	M1
UC 3	Analyser la traçabilité	Le moteur vérifie les liens entre SSS, SRS et SDD et attribue un statut à chaque exigence.	M2
UC 4	Consulter les anomalies	L'utilisateur visualise les exigences non couvertes, orphelines ou avec liens brisés.	M2
UC 5	Consulter le taux de couverture	L'utilisateur consulte les taux SSS/SRS, SRS/SDD, global et le taux de traçabilité complète SSS→SRS→SDD.	M2
UC 6	Visualiser le graphe	L'interface affiche les fichiers comme nœuds colorés par type, reliés par des liens avec pourcentage de couverture.	M3
UC 7	Filtrer les résultats	L'utilisateur filtre par statut, état développé, identifiant (autocomplétion) ou type de lien (brisé/orpheline).	M3
UC 8	Générer un rapport	L'outil exporte les résultats en CSV, HTML interactif ou PDF.	M3
UC 9	Démarrer l'application	L'utilisateur voit l'écran Accueil avec barre de progression animée, puis clique sur Démarrer.	M3

Tableau 3 – Description des cas d'utilisation.

Les relations «include» indiquent qu'un cas d'utilisation en déclenche automatiquement un autre : par exemple, UC1 déclenche toujours UC2, qui déclenche UC3.

Les relations «extend» indiquent qu'un cas d'utilisation peut optionnellement en déclencher un autre selon le choix de l'utilisateur : par exemple, après UC3, l'utilisateur peut choisir UC6, UC7 ou UC8.

4.2. Diagramme d'activité :

Le diagramme d'activité ci-dessous décrit le flux d'exécution complet de l'outil, de la sélection des fichiers par l'utilisateur jusqu'à l'affichage des résultats.

Il est organisé en couloirs, chaque couloir correspondant à un acteur ou un module.

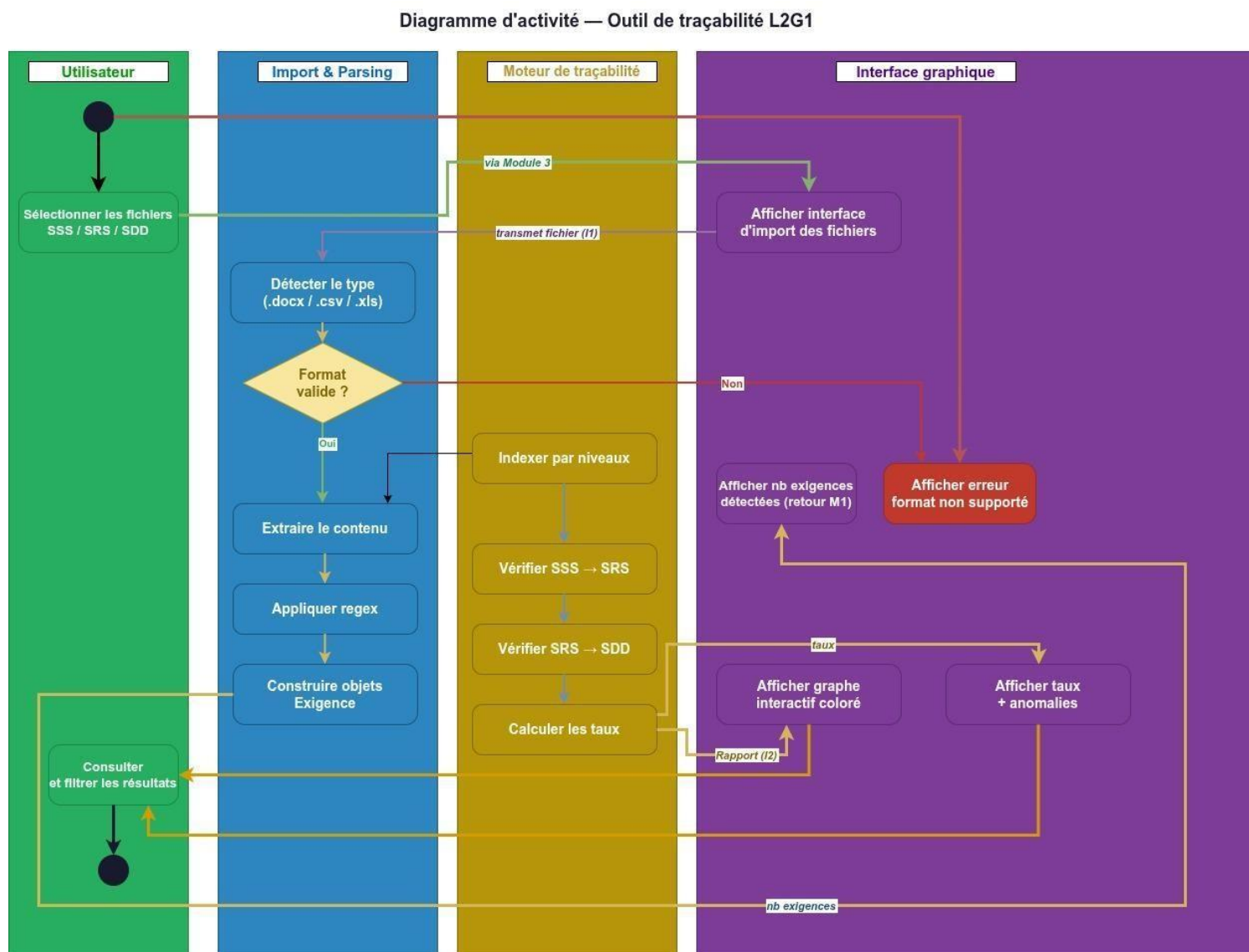


Figure 4 – Diagramme d'activité organisé par module

Le diagramme se lit de haut en bas. On distingue quatre couloirs :

- Utilisateur (vert) : sélectionne les fichiers au départ, reçoit les feedbacks intermédiaires du Module 1, et consulte les résultats finaux affichés par le Module 3.
- Module 1 – Import & Parsing (bleu) : détecte le type de fichier, lit son contenu, applique le regex, et construit les objets Exigence.
- Module 2 — Moteur de traçabilité (jaune) : indexe les exigences par niveau, vérifie les liens SSS→SRS puis SRS→SDD, et calcule les taux de couverture.
- Module 3 – Interface graphique (violet) : reçoit le RapportTracabilite et l'affiche sous forme de graphe coloré avec les taux et anomalies.

Deux comportements particuliers méritent d'être signalés. D'abord, une boucle de retour est prévue en cas de format de fichier non supporté : le Module 1 – Import & Parsing affiche un message d'erreur explicite et l'utilisateur est invité à choisir un autre fichier, sans que l'exécution ne s'interrompt brutalement. Ensuite, le Module 1 – Import & Parsing envoie un retour visuel à l'utilisateur après chaque import réussi, indiquant le nombre d'exigences détectées, ce qui permet à l'utilisateur de vérifier rapidement que l'import s'est bien déroulé.

5. Description des modules :

5.1. Module 1 : Import & Parsing :

5.1.1. Rôle

Le Module 1 — Import/Parsing est le point d'entrée unique de l'outil. Son rôle est de lire les fichiers fournis par l'utilisateur et de les transformer en objets C++ homogènes, quelle que soit leur origine.

Après ce module, le reste du système n'a plus à se préoccuper du format des fichiers sources : il travaille uniquement avec des objets Exigence standardisés.

Cette séparation est importante car si l'on souhaite ajouter un nouveau format de fichier dans le futur (par exemple le PDF), il suffira de modifier uniquement ce module sans toucher aux autres.

5.1.2. Fonctionnement

Le traitement se déroule en quatre étapes successives.

5.1.2.1. Détection du type de fichier par extension

La première étape consiste à identifier le format du fichier fourni par l'utilisateur en lisant son extension : .docx, .csv, .xls, .xlsx ou .doc. Les formats .doc et .xls sont convertis automatiquement en .docx/.xlsx via un appel système à LibreOffice avant traitement. Si l'extension n'est pas reconnue (.pdf, .txt, etc.), le module déclenche une exception `FileFormatException` sans tenter de lire le fichier.

Cette détection est indispensable car chaque format implique une méthode de lecture radicalement différente : un .docx est une archive ZIP contenant du XML, un .csv est un fichier texte brut avec des séparateurs, et un .xls est un format binaire propriétaire.

Sans cette détection préalable, le module ne saurait pas quelle bibliothèque ou quelle logique appliquer. Si l'extension n'est pas reconnue (.pdf, .txt, etc.), le module déclenche immédiatement une erreur de type "Format non supporté" sans tenter de lire le fichier.

5.1.2.2. Extraction du contenu

Une fois le type détecté, le module extrait le texte brut du fichier. Cette étape est nécessaire car les exigences sont enfouies dans des structures de fichier complexes. Il faut les "déballer" avant de pouvoir les analyser.

Pour un fichier .docx : le fichier est en réalité une archive ZIP. Le module le décompresse en mémoire (sans écriture sur le disque) grâce à zlib/minizip, puis lit le fichier interne word/document.xml. Le texte des paragraphes se trouve dans les balises <w:t>.

Exemple de structure XML extraite :

```
<w:p>
  <w:r>
    <w:t>EXIG_CLIENT_01 : Le système doit importer les fichiers.</w:t>
  </w:r>
</w:p>
```

Pour un fichier .csv : le fichier est lu ligne par ligne. Chaque ligne est découpée selon le délimiteur (; ou ,).

Exemple de contenu parsé :

```
ID;Description;Lien
EXIG_LOG_01;Lecture .docx via ZIP;EXIG_CLIENT_01
EXIG_LOG_02;Parsing CSV délimiteur ',';EXIG_CLIENT_02
```

Pour un fichier .xls : la structure binaire est lue cellule par cellule, le contenu textuel de chaque cellule est extrait de la même façon qu'un CSV.

5.1.2.3. Application du pattern regex

Une fois le texte extrait, le module applique une expression régulière configurable pour repérer les identifiants d'exigences. Cette étape est nécessaire car les exigences ne sont pas toujours présentées de façon identique d'un document à l'autre. Seul un pattern générique permet de les détecter automatiquement quelle que soit la mise en forme.

Le pattern par défaut est par exemple :

```
[A-Z] [A-Z0-9_] {2, }
```

Appliqué sur la ligne `EXIG_CLIENT_01 : Le système doit importer les fichiers`, il détecte `EXIG_CLIENT_01` comme identifiant.

Le pattern est configurable pour s'adapter aux conventions de nommage propres au projet.

5.1.2.4. Construction des objets Exigence

Pour chaque identifiant détecté, le module construit un objet C++ de type `Exigence` (voir section 7.1).

Cela permet de standardiser la représentation des données indépendamment du format d'origine.

Après cette étape, le reste du système ne manipule que des objets homogènes, sans se préoccuper de savoir si l'exigence venait d'un `.docx` ou d'un `.csv`. Exemple d'objet construit :

```
Exigence {  
    id          = "EXIG_CLIENT_01"  
    contenu     = "Le système doit importer les fichiers."  
    typeDocument = SSS  
    fichierSource = "sss_client.docx"  
    ligneSource  = 42  
}
```

À l'issue de cette étape, tous les objets `Exigence` sont regroupés dans une liste transmise au Module 2 – Moteur de traçabilité via l'interface I1.

5.1.3. Gestion des erreurs

5.1.3.1. Fichier introuvable :

Le Module 1 vérifie l'existence du fichier avant toute opération.

Si le chemin fourni est invalide ou le fichier absent du disque, le Module 3 – Interface graphique affiche le message « Fichier introuvable » et l'utilisateur est invité à sélectionner un nouveau fichier.

L'outil ne plante pas : il boucle proprement vers l'étape de sélection.

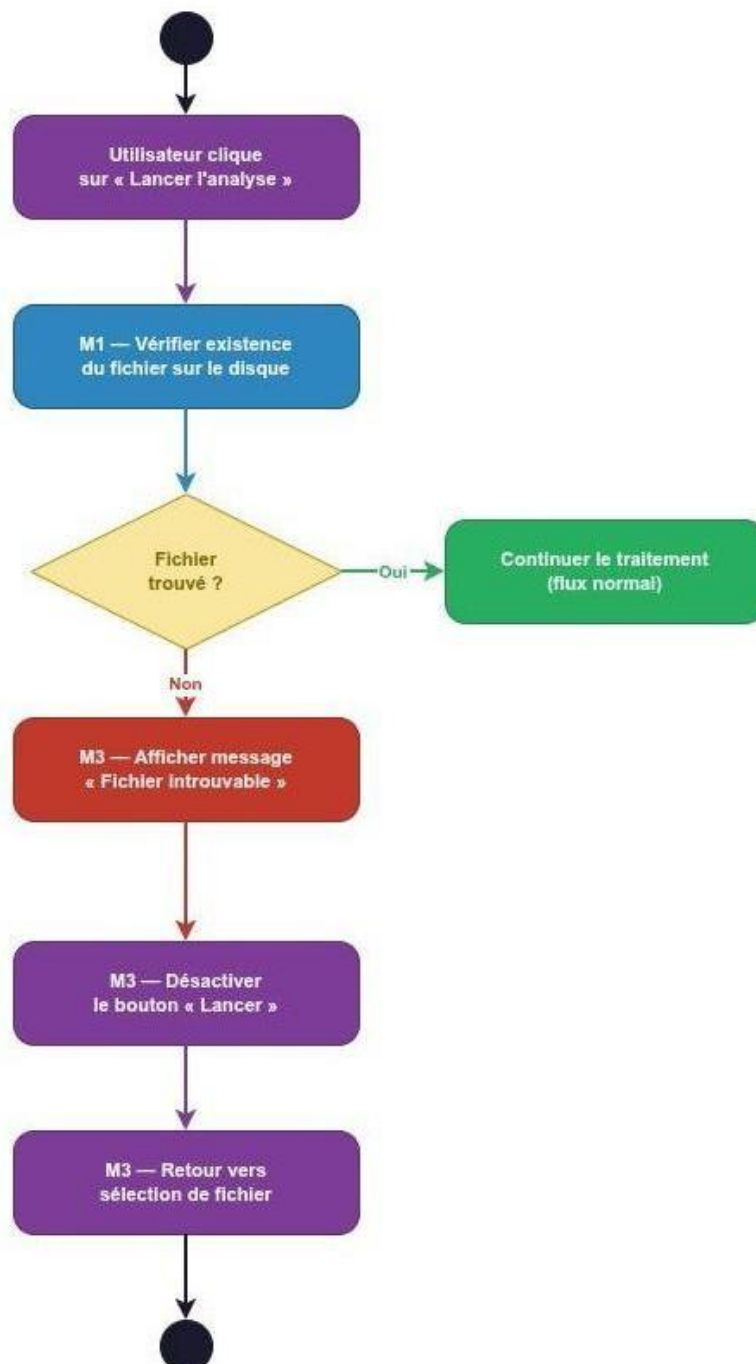


Figure 5 – Gestion des fichiers introuvables.

5.1.3.2. Fichier corrompu :

Si le fichier existe mais que sa structure interne est invalide (archive .docx mal formée, XML mal structuré), la décompression ou le parsing échoue.

Le Module 3 – Interface graphique affiche « Fichier illisible ou corrompu » et l'utilisateur est invité à fournir un fichier valide.

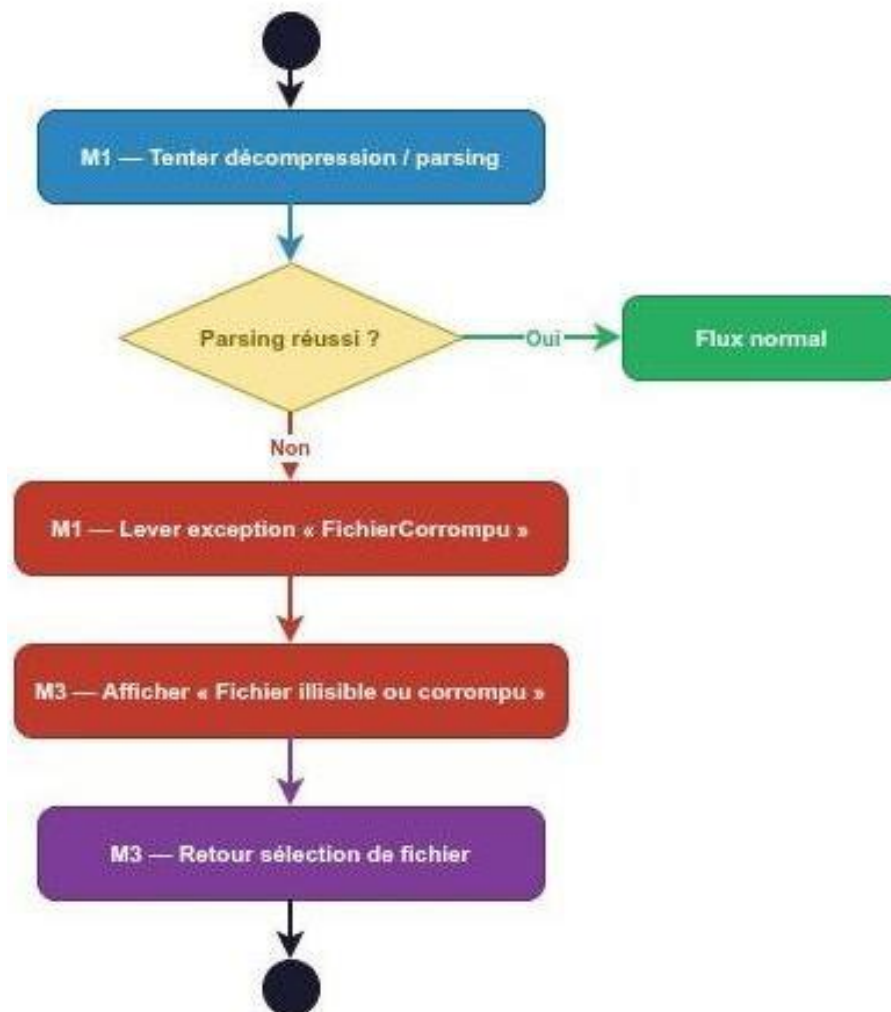


Figure 6 – Gestion des fichiers corrompus.

5.1.3.3. Fichier non supporté :

Si l'extension du fichier n'est pas reconnue (.pdf, .txt, etc.), le Module 1 – Import & Parsing arrête immédiatement le traitement sans tenter de lire le fichier.

Le Module 3 – Interface graphique affiche « Format non supporté » avec la liste des formats acceptés : .docx, .csv, .xls.

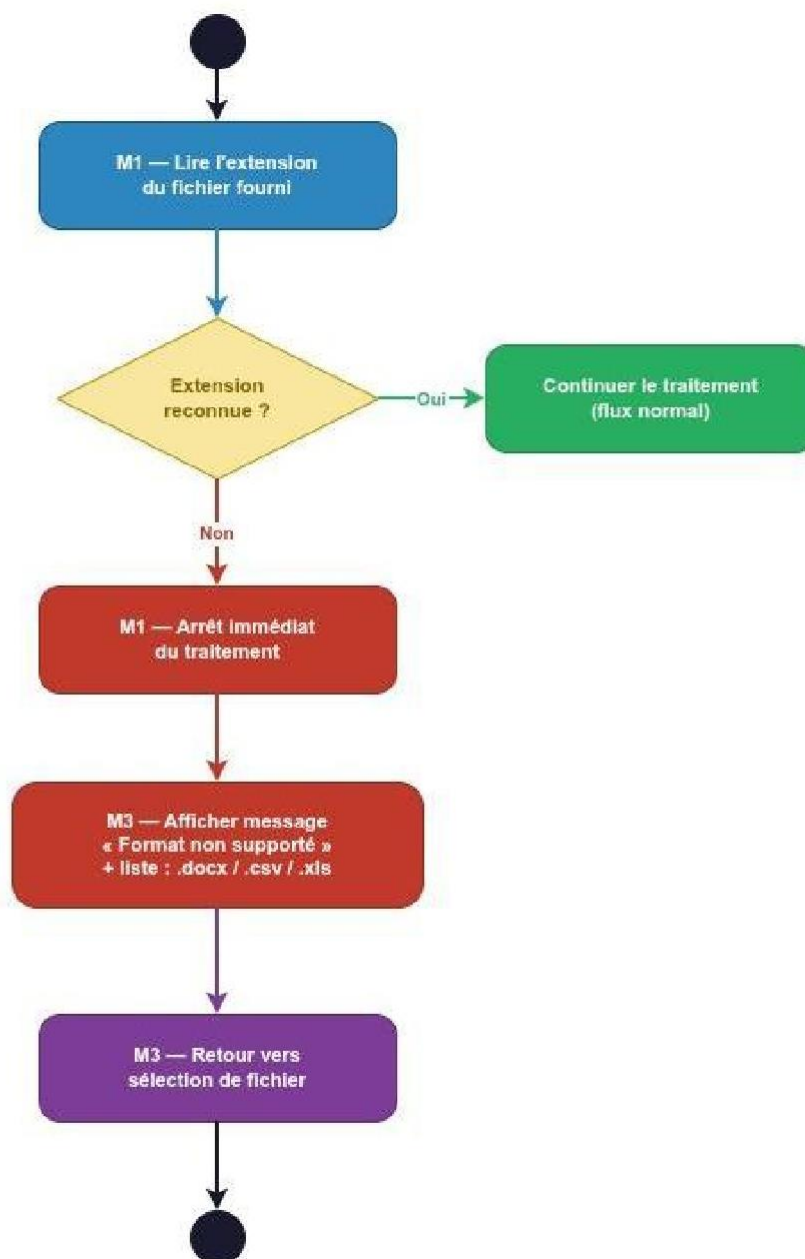


Figure 7 – Gestion des fichiers non supportés.

5.2. Module 2 : Moteur de traçabilité :

5.2.1. Rôle :

Le Module 2 – Moteur de traçabilité reçoit la liste d'objets Exigence produite par le Module 1 – Import & Parsing et réalise l'analyse de traçabilité.

Son travail consiste à vérifier que chaque exigence d'un niveau est bien couverte par au moins une exigence du niveau suivant et à calculer les taux de couverture correspondants.

C'est-à-dire que pour chaque exigence SSS, le moteur vérifie qu'au moins une exigence SRS y fait référence. De même, pour chaque exigence SRS, il vérifie qu'au moins un élément SDD la couvre.

Par exemple, si l'exigence **EXIG_CLIENT_01** du SSS n'est référencée par aucune exigence SRS, elle sera marquée **NON_COUVERTE** et apparaîtra en rouge dans l'interface.

5.2.2. Fonctionnement :

L'analyse se déroule en cinq étapes.

5.2.2.1. Indexation

Avant toute vérification, le moteur construit des tables de recherche rapide (`std::map` en C++) qui associent chaque identifiant d'exigence à son objet correspondant.

Cette étape est indispensable car sans index, retrouver une exigence par son identifiant nécessiterait de parcourir toute la liste à chaque fois, ce qui serait trop lent pour de grands projets. Avec une `std::map`, la recherche se fait en temps constant.

Exemple de structure construite :

index_SSS["EXIG_CLIENT_01"] → objet Exigence { id, contenu, statut... }

index_SSS["EXIG_CLIENT_02"] → objet Exigence { ... }

index_SRS["EXIG_LOG_01"] → objet Exigence { ... }

5.2.2.2. Vérification SSS → SRS

Le moteur parcourt toutes les exigences SRS et consulte leur champ `liensTracabilite`.

Ce champ contient la liste des identifiants SSS que cette exigence SRS déclare couvrir. Pour chaque identifiant SSS listé, le moteur vérifie qu'il existe bien dans l'index SSS.

Si l'identifiant est introuvable, cela signifie que le lien est brisé (il pointe vers une exigence qui n'existe pas). Un objet `LienBrise` est alors créé.

Exemple :

```
EXIG_LOG_01.liensTracabilite = ["EXIG_CLIENT_01", "EXIG_CLIENT_99"]
```

→ **EXIG_CLIENT_01 trouvé dans index_SSS**

→ **EXIG_CLIENT_99 introuvable dans index_SSS → LienBrise créé**

5.2.2.3. Vérification SRS → SDD

Le même principe est appliqué entre les niveaux SRS et SDD.

Le moteur parcourt les éléments SDD, consulte leurs `liensTracabilite` vers le SRS, et vérifie que chaque identifiant SRS référencé existe bien dans l'index SRS.

Exemple :

```
CONCEPTION_01.liensTracabilite = ["EXIG_LOG_01", "EXIG_LOG_02"]
```

→ **EXIG_LOG_01 trouvé dans index_SRS**

→ **EXIG_LOG_02 trouvé dans index_SRS**

5.2.2.4. Attribution du statut

À l'issue des vérifications, chaque exigence reçoit un `StatutTracabilite` (voir section 7.2) :

- `COUVERTE` : au moins une exigence du niveau inférieur ne la référence : affichée en vert.
- `NON_COUVERTE` : aucune exigence du niveau inférieur ne la couvre : affichée en rouge.
- `LIEN_BRISE` : elle est référencée mais l'identifiant cible est introuvable : affichée en orange.

Exemple :

À l'issue des vérifications, chaque exigence reçoit un statut : `COUVERTE` (vert) – au moins une exigence du niveau inférieur la référence ; `NON_COUVERTE` (rouge) – aucune exigence du niveau inférieur ne la couvre ; `LIEN_BRISE` (orange) – référencée mais l'identifiant cible est introuvable ; `ORPHELINE` (violet) – SRS sans aucune référence SSS, ou SDD sans aucune référence SRS.

5.2.2.5. Calcul des taux

Le moteur calcule les trois taux de couverture définis dans le cahier des charges. Ces taux permettent à l'utilisateur d'évaluer en un coup d'œil l'état global de la traçabilité du projet :

$$\begin{aligned}
 \text{Taux}_{\frac{SSS}{SRS}} &= \frac{\text{nb exigences SSS COUVERTES}}{\text{nb total exigences SSS}} \\
 \text{Taux}_{\frac{SRS}{SDD}} &= \frac{\text{nb exigences SRS COUVERTES}}{\text{nb total exigences SRS}} \\
 \text{TauxGlobal} &= \text{Taux}_{\frac{SSS}{SRS}} \times \text{Taux}_{\frac{SRS}{SDD}}
 \end{aligned}$$

L'ensemble des résultats est regroupé dans un objet `RapportTracabilite` transmis au Module 3 – Interface graphique via l'interface I2.

5.2.3. Calcul du taux de couverture :

Les taux de couverture sont définis comme suit :

$$\begin{aligned}
 \text{Taux} \frac{SSS}{SRS} &= \frac{\text{nb exigences SSS COUVERTES}}{\text{nb total exigences SSS}} \\
 \text{Taux} \frac{SRS}{SDD} &= \frac{\text{nb exigences SRS COUVERTES}}{\text{nb total exigences SRS}} \\
 \text{TauxGlobal} &= \text{Taux} \frac{SSS}{SRS} \times \text{Taux} \frac{SRS}{SDD}
 \end{aligned}$$

Par exemple: si 9 exigences SSS sur 10 sont couvertes ($\text{TauxSSS}/\text{SRS} = 0,90$) et 8 exigences SRS sur 9 sont couvertes ($\text{TauxSRS}/\text{SDD} = 0,89$), alors le taux global vaut $0,90 \times 0,89 = 0,80$, soit 80 %.

5.3. Module 3 : Interface graphique :

5.3.1. Rôle :

Le Module 3 – Interface Graphique reçoit le Rapport de Traçabilité produit par le Module 2 – Moteur de traçabilité et en affiche le contenu à l'utilisateur de manière visuelle et interactive.

Son objectif est de permettre à l'utilisateur d'explorer les résultats rapidement, d'identifier les anomalies et de filtrer les données selon ses besoins.

5.3.2. Fonctionnalités :

Le Module 3 – Interface Graphique propose les fonctionnalités suivantes :

5.3.2.1. Graphe interactif

Les exigences sont représentées sous forme de nœuds reliés par des branches représentant les liens de traçabilité entre niveaux.

Cette représentation graphique est essentielle car elle permet à l'utilisateur de visualiser d'un seul coup d'œil l'ensemble des relations $SSS \rightarrow SRS \rightarrow SDD$, ce qu'un simple tableau ne permettrait pas.

Chaque nœud correspond à une exigence, chaque branche à un lien de traçabilité déclaré. L'utilisateur peut naviguer dans ce graphe, zoomer, et cliquer sur un nœud pour afficher le détail de l'exigence correspondante.

5.3.2.2. Code couleur

Chaque nœud du graphe est coloré selon le statut attribué par le Module 2 – Moteur de traçabilité :

- **Vert** → COUVERTE : l'exigence est bien référencée par au moins une exigence du niveau inférieur.
- **Rouge** → NON_COUVERTE : aucune exigence du niveau inférieur ne la couvre. Cela indique un besoin client potentiellement oublié.

Chaque nœud du graphe est coloré selon le statut attribué par le Module 2 : Vert → COUVERTE ; Rouge → NON_COUVERTE ; Orange → LIEN_BRISE (référence vers un identifiant inexistant) ; Violet → ORPHELINE – SRS sans référence SSS ou SDD sans référence SRS.

Ce codage permet une lecture visuelle immédiate sans avoir à lire le détail de chaque exigence – l'utilisateur repère instantanément les zones problématiques.

5.3.2.3. Affichage des taux

Les trois taux de couverture calculés par le Module 2 – Moteur de traçabilité sont affichés de façon permanente dans l'interface, mis à jour à chaque analyse :

- **TauxSSS/SRS** : pourcentage d'exigences SSS couvertes par au moins une exigence SRS.
- **TauxSRS/SDD** : pourcentage d'exigences SRS couvertes par au moins un élément SDD.

Les taux affichés en permanence dans l'interface : TauxSSS/SRS – pourcentage d'exigences SSS couvertes par au moins une SRS ; TauxSRS/SDD – pourcentage d'exigences SRS couvertes par au moins un élément SDD ; TauxGlobal – produit des deux taux intermédiaires ; TauxTracabiliteComplete – proportion de SSS dont la chaîne SSS→SRS→SDD est intégralement couverte.

5.3.2.4. Filtrage

L'utilisateur peut affiner l'affichage du graphe en appliquant des filtres. Cette fonctionnalité est utile sur de grands projets où le graphe complet devient illisible. Trois types de filtres sont disponibles :

- **Par document source** : afficher uniquement les exigences SSS, SRS ou SDD.
- **Par statut** : afficher uniquement les exigences COUVERTE, NON_COUVERTE ou LIEN_BRISE.
- **Par identifiant** : rechercher et isoler une exigence spécifique par son ID.

Ces filtres sont transmis au Module 2 – Moteur de traçabilité via l'interface I3 pour recalcul dynamique.

5.3.2.5. Génération de rapport

L'outil produit un rapport exportable en trois formats : CSV (tableau structuré), HTML interactif (filtres JavaScript et recherche temps réel), ou PDF (via QPrinter). Il inclut la liste complète des exigences avec leur statut, les liens brisés, les orphelines et les quatre taux de couverture.

Ce rapport permet à l'utilisateur de conserver une trace écrite de l'état de la traçabilité à un instant donné, et de le partager avec d'autres membres du projet. Le rapport inclut la liste complète des exigences avec leur statut, la liste des liens brisés, et les trois taux de couverture.

6. Interfaces entres modules :

Les interfaces désignent les points de communication entre deux modules et définissent quelles données sont échangées.

Dans notre architecture, trois interfaces sont identifiées :

Interf ace	De	Vers	Données transmises	Condition de déclenchement
I1	Modul e 1	Modul e 2	Vecteurs ExigenceSSS, ExigenceSRS, ExigenceSDD (via ConvertisseurExigences)	Import réussi.
I2	Modul e 2	Modul e 3	RapportTracabilite (taux SSS→SRS, SRS→SDD, global, complet, liensBrises, orphelines)	Analyse complète effectuée.
I3	Modul e 3	Modul e 2	Filtres utilisateur (statut, état développé, identifiant, type lien)	L'utilisateur applique un filtre.

Tableau 4 – Interfaces entre les modules, données échangées et conditions.

Le schéma ci-dessus illustre les trois interfaces de communication entre les modules.

Les données circulent de façon séquentielle de gauche à droite : le Module 1 – Import & Parsing transmet via I1 la liste des objets Exigence au Module 2 – Moteur de traçabilité dès que l'import est réussi.

Le Module 2 transmet ensuite via I2 l'objet RapportTracabilite au Module 3 – Interface graphique une fois l'analyse complète effectuée.

L'interface I3 représente le seul flux retour du système : lorsque l'utilisateur applique un filtre dans l'interface, le Module 3 interroge le Module 2 pour recalculer dynamiquement les résultats à afficher.

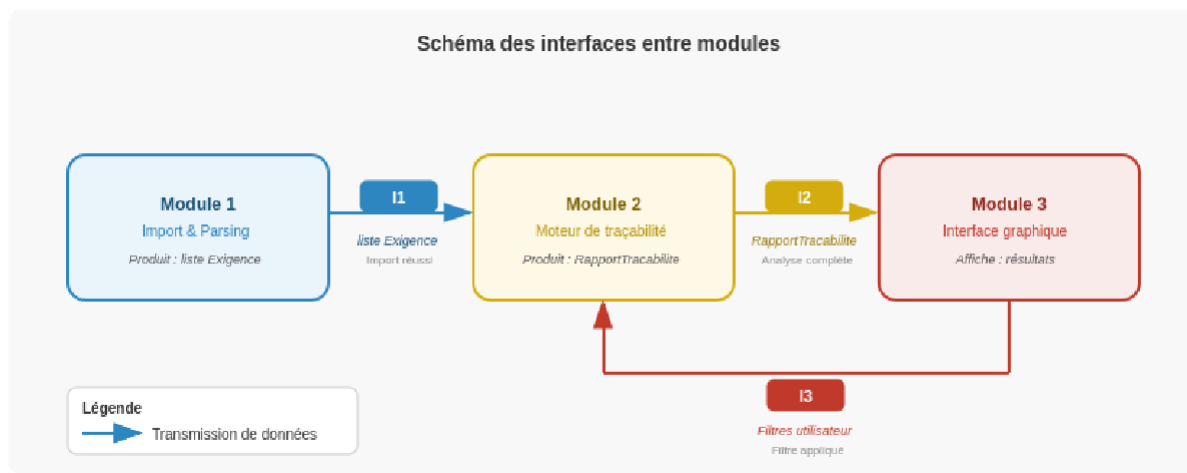


Figure 8 – Schéma des interfaces entres modules.

7. Modèle de données :

Le modèle de données décrit les structures utilisées dans le programme pour stocker et faire circuler les données entre les modules.

On distingue quatre structures principales : Exigence, StatutTracabilite, LienBrise, et RapportTracabilite.

7.1. Structure Exigence :

La structure Exigence est la brique de base du programme. Elle représente une seule exigence extraite d'un fichier. Le Module 1 – Import & Parsing crée un objet de ce type pour chaque exigence détectée. Elle contient six champs.

Le champ ID stocke l'identifiant unique de l'exigence, c'est le champ le plus important.

Le champ contenu stocke le texte brut de l'exigence tel qu'il apparaît dans le fichier. Le champ liensTracabilite est une liste d'identifiants : ce sont les exigences du niveau supérieur que cette exigence déclare couvrir. Par exemple, une exigence SRS qui couvre EXIGENCE_CLIENT_VOIE_1 aura cet identifiant dans sa liste.

Le champ typeDocument indique si l'exigence vient d'un fichier SSS, SRS ou SDD. Enfin, les champs fichierSource et ligneSource permettent de localiser précisément l'exigence dans le fichier d'origine, utile pour afficher des messages d'erreur clairs à l'utilisateur.

7.2. Enumeration StatutTarcabilite :

Le StatutTracabilite est attribué par le Module 2 – Moteur de traçabilité à chaque exigence à l'issue de l'analyse.

Il peut prendre trois valeurs, résumées dans le tableau ci-dessous. Ce statut est directement utilisé par le Module 3 – Interface graphique pour colorier les nœuds du graphe interactif, permettant à l'utilisateur d'identifier visuellement et instantanément l'état de chaque exigence.

Valeur	Signification	Couleur
COUVERTE	L'exigence est référencée par au moins une exigence du niveau inférieur.	Vert
NON_COUVERTE	Aucune exigence du niveau inférieur ne la couvre. Indique un besoin potentiellement oublié.	Rouge
LIEN_BRISE	L'exigence fait référence à un identifiant qui n'existe pas dans le document cible.	Orange
ORPHELINE	SRS sans référence SSS, ou SDD sans référence	Violet

	SRS. Panneau violet dédié dans l'interface.	t
--	---	---

7.3. Structure LienBrise :

Lorsque le Module 2 – Moteur de traçabilité détecte un lien brisé lors de l'analyse, il crée un objet LienBrise afin de conserver toutes les informations nécessaires pour signaler l'erreur précisément à l'utilisateur.

Les quatre champs de cet objet sont décrits dans le tableau ci-dessous. Ces informations sont transmises au Module 3 – Interface graphique via l'interface I2 dans le RapportTracabilite, et permettent d'afficher un message d'erreur ciblé indiquant exactement quel fichier, quelle exigence et quel sens de lien est concerné.

<u>Champs</u>	<u>Type</u>	<u>Description</u>	<u>Exemple</u>
idSource	String	Identifiant de l'exigence qui contient le lien invalide.	EXIG_LOG_01
idCible	String	Identifiant introuvable dans le document cible.	EXIG_CLIENT_99
fichierSource	String	Nom du fichier où se trouve l'erreur.	srs_projet.csv
direction	enum	Sens du lien concerné.	SSS→SRS ou SRS→SDD

7.4. Structure RapportTracabilite :

Le RapportTracabilite est l'objet produit par le Module 2 – Moteur de traçabilité à la fin de son analyse et transmis au Module 3 – Interface graphique via l'interface I2 pour affichage. Il regroupe en un seul objet tout ce dont le Module 3 a besoin pour afficher les résultats. Sa structure est la suivante :

Champ	Type C++	Description	Exemple
exigencesSSS	vector<ExigenceSSS>	Liste des ExigenceSSS avec statut attribué.	id:"EXIG_CLIENT_01", statut:COUVERTE
exigencesSRS	vector<ExigenceSRS>	Liste des ExigenceSRS avec statut attribué.	id:"EXIG_LOG_01", statut:LIEN_BRISE
exigencesSDD	vector<ExigenceSDD>	Liste des ExigenceSDD avec statut attribué.	id:"CONCEPTION_01", statut:COUVERTE
liensBrises	vector<string>	Liste des liens brisés avec nomFichier source inclus.	"EXIG_LOG_01 [srs.docx] → EXIG_CLIENT_99"
orphelines	vector<string>	Liste des orphelines (SRS sans SSS ou SDD sans SRS), séparée des liens brisés.	"Orpheline SRS: EXIG_LOG_05"
tauxSSS_SRS	double	Taux de couverture SSS→SRS, entre 0.0 et 1.0.	0.80
tauxSRS_SDD	double	Taux de couverture SRS→SDD, entre 0.0 et 1.0.	0.90
tauxGlobal	double	Produit des deux taux intermédiaires.	0.72
tauxTracabiliteComplete	double	Proportion de SSS dont la chaîne SSS→SRS→SDD est intégralement couverte.	0.80

8. Prise en compte des contraintes non fonctionnelles :

Les contraintes non fonctionnelles définissent des exigences sur la qualité du système, indépendamment de ce qu'il fait. Elles portent sur ses performances, sa portabilité, sa sécurité et sa maintenabilité. Ces contraintes sont directement issues du cahier des charges.

8.1. Performance :

L'outil sera capable d'analyser 100 exigences en moins de 5 secondes. Il utilise des tables d'indexation (std::map) pour les recherches par identifiant.

8.2. Portabilité :

L'outil fonctionne sur Windows, Linux et macOS. Il utilise de bibliothèques multiplateformes ([A6] Qt/wxWidgets, [A4] zlib, [A5] tinyxml2) et de compilateurs compatibles : GCC, MinGW, Clang.

8.3. Confidentialité :

L'outil a un fonctionnement 100 % local et ne nécessite aucune connexion réseau. Aucune API externe ou accès réseau est nécessaire dans le code.

Tout le traitement est effectué en mémoire sur la machine de l'utilisateur.

8.4. Maintenabilité :

Le code est modulaire, commenté et lisible.

Les responsabilités sont réparties sur les trois modules et chaque module aura son propre répertoire et son fichier README décrivant son rôle.

9. Glossaire :

Terme	Définition
SSS (System/Software Specification)	Document d'exigences rédigé par le client. Il décrit ce que le système doit faire, sans préciser comment. C'est le niveau le plus haut de la hiérarchie des exigences.
SRS (Software Requirements Specification)	Document de spécifications techniques rédigé par l'équipe de développement. Il traduit les exigences du SSS en termes techniques. Chaque exigence SRS doit être reliée à au moins une exigence SSS.
SDD (Software Design Document)	Document de conception décrivant l'implémentation technique. Chaque élément du SDD doit être relié à au moins une exigence SRS.
Traçabilité	Capacité à établir et vérifier des liens entre les exigences de différents niveaux documentaires. Une bonne traçabilité garantit que chaque besoin client se retrouve dans la conception finale.
Exigence couverte	Exigence d'un niveau donné qui est référencée par au moins une exigence du niveau inférieur. Statut COUVERTE, affiché en vert.
Exigence non couverte	Exigence d'un niveau donné qu'aucune exigence du niveau inférieur ne référence. Statut NON_COUVERTE, affiché en rouge. Indique un besoin client potentiellement oublié.
Exigence orpheline	Exigence SRS ou SDD qui ne référence aucune exigence du niveau supérieur. Elle existe sans raison tracée dans le projet.
Lien brisé	Référence dans le champ liensTracabilite d'une exigence vers un identifiant qui n'existe pas dans le document cible. Statut LIEN_BRISE, affiché en orange.
Regex / Expression régulière	Séquence de caractères définissant un motif de recherche dans un texte. Dans notre outil, elle est utilisée pour détecter automatiquement les identifiants d'exigences dans les fichiers importés.
Parsing	Processus d'analyse syntaxique d'un fichier pour en extraire des informations structurées. Dans notre outil, le parsing consiste à extraire les identifiants et le texte des exigences.
API	Interface de Programmation Applicative. Service externe auquel un programme peut se connecter via le réseau pour obtenir des données ou des fonctionnalités.
STL (Standard Template Library)	Bibliothèque standard du C++ fournissant des structures de données (vecteurs, maps, listes) et des algorithmes prêts à l'emploi.

Terme	Définition
Open source	Logiciel dont le code source est accessible et librement utilisable, modifiable et redistribuable.
Exigence orpheline SRS	SRS qui ne référence aucune exigence SSS (tracabilite vide). Distincte d'un lien brisé. Affichée en violet.
Exigence orpheline SDD	SDD qui ne référence aucune exigence SRS (referencedSRS vide). Affichée en violet.
TauxTracabiliteComplete	Proportion de SSS dont la chaîne SSS→SRS→SDD est intégralement couverte par une SRS elle-même couverte par une SDD.
ConvertisseurExigences	Classe du Module 1 transformant les ExigenceExtraite en objets typés ExigenceSSS/SRS/SDD selon des préfixes configurables par fichier.
LibreOffice (appel système)	Utilisé pour convertir .doc et .xls en .docx/.xlsx avant extraction.
Accueil (splash screen)	Fenêtre de démarrage Qt avec barre de progression animée. S'affiche avant MainWindow.
Export HTML interactif	Page HTML avec filtres JavaScript et recherche temps réel, produite par OngletRapport.

10. Index :

Terme	Type	Sections
Exigence	Structure de données C++	3.1, 5.1, 6, 7.1
StatutTracabilite	Énumération C++	5.2, 7.2
LienBrise	Structure de données C++	5.2, 7.3
RapportTracabilite	Structure de données C++	3.1, 5.2, 6, 7.4
TypeDocument	Énumération C++	7.1
COUVERTE	Valeur de StatutTracabilite	7.2
NON_COUVERTE	Valeur de StatutTracabilite	7.2
LIEN_BRISE	Valeur de StatutTracabilite	7.2
Module 1 – Import & Parsing	Module logiciel	3.1, 4.2, 5.1, 6
Module 2 – Moteur de traçabilité	Module logiciel	3.1, 4.2, 5.2, 6
Module 3 – Interface graphique	Module logiciel	3.1, 4.2, 5.3, 6
Interface I1	Interface inter-module	6
Interface I2	Interface inter-module	6
Interface I3	Interface inter-module	6
SSS	Type de document	2, 3.1, 5.2, 7.1, 7.4, 10
SRS	Type de document	2, 3.1, 5.2, 7.1, 7.4, 10
SDD	Type de document	2, 3.1, 5.2, 7.1, 7.4, 10
Regex / Expression régulière	Technique de traitement de texte	2, 3.2, 4.1, 5.1, 10
std::map [A3]	Structure STL C++	5.2, 8
std::regex [A3]	Bibliothèque STL C++	3.2

Terme	Type	Sections
zlib / minizip [A4]	Bibliothèque externe	3.2, 5.1
tinyxml2 / pugixml	Bibliothèque externe	3.2, 5.1
Qt / wxWidgets [A6]	Bibliothèque graphique	3.2, 5.3

11. Références :

[A1] - Cahier des Charges du projet L2G1.

[A2] - Cahier de Recettes du projet L2G1.

[A3] - Documentation C++ STL, disponible sur cppreference.com.

[A4] - Documentation de la bibliothèque zlib et minizip, disponible sur zlib.net.

[A5] - Documentation de la bibliothèque tinyclang.

[A6] - Documentation Qt.

12. Suivi des modifications :

Cette section recense l'ensemble des modifications apportées au cahier des recettes au fil des versions.

12.1. Version A1 :

Première version de la conception générale.

Le tableau ci-dessous liste les demandes traitées, identifiées par leur ID sur la forge, avec le titre de la demande et le détail des modifications effectuées (ajout de chapitre, d'exigence, modification de contenu, etc.).

<u>ID</u>	<u>Titre</u>	<u>Commentaire</u>
5863	Modification du diagramme de cas d'utilisation	Création d'un diagramme des cas d'utilisation plus lisible, en séparant chaque module.
5864	Modification du diagramme d'activité	Retracer les interaction utilisateur-interface graphique
5865	Détailler la description du fonctionnement du module 1 - Import & Parsing	Détailler chaque tiret en sous-sections 5.1.2.1 à 5.1.2.4 avec pourquoi, comment, exemples
5866	Amélioration de la section gestion des erreurs	faire un diagramme par exception avec sa propre sous-section 5.1.3.1 / 5.1.3.2 / 5.1.3.3
5867	Amélioration du module 2 - Moteur de traçabilité	Ajouter des explications concrètes dans le rôle du module et détailler le fonctionnement
5868	Création d'un schéma de liens dans la section 6	Ajouter un schéma des liens I1 / I2 / I3 avec phrase d'accompagnement
6006	Amélioration du diagramme d'activité	Retracer les éléments et flèches en sorte que le diagramme soit plus lisible
6007	Amélioration des diagrammes de gestion d'erreurs	Retracer les diagramme de gestion d'erreur d'import de fichiers en sorte qu'ils soient plus lisibles
7386	MÀJ Tableau 1 – Vue d'ensemble	Sorties M1/M2 mises à jour :

		ExigenceSSS/SRS/SDD, RapportTracabilite étendu (orphelines, taux complet).
7387	MÀJ Tableau 2 – Choix techniques	Ajout .xlsx (libzip+XML), .doc/.xls (LibreOffice), threads atomic<bool>. Qt confirmé.
7388	MÀJ Tableau 3 – Cas d'utilisation	Ajout UC9 (Accueil). Mise à jour UC1, UC4, UC5, UC7, UC8.
7389	MÀJ Tableau 4 – Interfaces	I1 : types ExigenceSSS/SRS/SDD. I2 : orphelines + tauxTracabiliteComplete. I3 : filtres étendus.
7390	MÀJ Module 1 – formats supportés	Ajout .xlsx (libzip+XML) et .doc/.xls (LibreOffice).
7391	MÀJ Module 1 – préfixes configurables	ConvertisseurExigences : préfixes multiples par virgule, configurables par fichier.
7392	MÀJ Module 2 – statut ORPHELINE	Nouveau statut ORPHELINE (violet) : SRS sans tracabilite ou SDD sans referencedSRS.
7394	MÀJ Module 2 – taux tracabilité complète	tauxTracabiliteComplete ajouté dans MoteurTracabilite et RapportTracabilite.
7395	MÀJ Module 3 – Accueil	Nouvelle classe Accueil avec barre de progression animée et signal demarrerClique().
7396	MÀJ Module 3 – code couleur violet	Panneau orphelines (violet) cliquable dans OngletRapport.
7397	MÀJ Module 3 – exports multi-format	Exports CSV, HTML interactif et PDF via QPrinter ajoutés.
7398	MÀJ StatutTracabilite	Ajout valeur ORPHELINE (violet).
7399	MÀJ RapportTracabilite	Ajout champs orphelines et tauxTracabiliteComplete.
7400	MÀJ Glossaire	Ajout : Orpheline SRS/SDD, TauxTracabiliteComplete, ConvertisseurExigences, LibreOffice, Accueil, Export HTML.

Version	Date	Émetteur	Valideur	Raisons
A1	17.03.26	Emilie Huang	Jérémy Meynadier	
A2	03.05.2026	Lamia Arrahmane	Jérémy Meynadier	